MagicBridge + LabVIEW Integration Package

📦 Contenido del Paquete

Este paquete contiene todo lo necesario para integrar el MagicBridge ELMOS Programmer con LabVIEW en Windows.

```
MagicBridge_LabVIEW_Integration/
├── docs/
│   └── INTEGRACION_LABVIEW.md      # 📘 Guía completa de integración
│
├── examples/
│   └── EJEMPLO_PASO_A_PASO.md      # 📝 Tutorial paso a paso
│
├── python_wrappers/
│   ├── detect_port.py              # 🔍 Detectar puerto COM
│   └── magicbridge_api.py          # 🔌 API wrapper simplificada
│
├── labview_vis/                    # 📊 VIs de ejemplo (*)
│   ├── DetectarMagicBridge.vi
│   ├── CargarDatosHMF.vi
│   ├── ProgramarChip.vi
│   └── MagicBridge_Main.vi
│
├── bin/                            # 🔧 Scripts compilados
│   └── (copiar desde MagicBridge_Protected_Package)
│
└── README.md                       # Este archivo

(*) Los VIs son plantillas/pseudocódigo. Requieren creación en LabVIEW.
```

🎯 Opciones de Integración

✅ Opción 1: System Exec (Recomendada)

- Simple y directa
- Compatible con todas las versiones de LabVIEW
- No requiere modificaciones

✅ Opción 2: Python Node

- LabVIEW 2018+
- Integración más profunda
- Mejor rendimiento

✅ Opción 3: DLL Wrapper

- Máxima velocidad
- Oculta implementación
- Más complejo

🚀 Inicio Rápido

1. Instalar Requisitos

```
REM Python 3.9+
python --version

REM pyserial
pip install pyserial
```

2. Copiar Archivos

```
REM Crear estructura
mkdir C:\MagicBridge\bin
mkdir C:\MagicBridge\helpers

REM Copiar scripts protegidos
xcopy bin C:\MagicBridge\bin\ /E

REM Copiar helpers
xcopy python_wrappers C:\MagicBridge\helpers\ /E
```

3. Verificar Instalación

```
REM Detectar puerto
python C:\MagicBridge\helpers\detect_port.py

REM Debe mostrar: COM3 (o el puerto correspondiente)
```

4. Crear VI en LabVIEW

Ver examples/EJEMPLO_PASO_A_PASO.md

📚 Documentación

Principal

- **docs/INTEGRACION_LABVIEW.md** - Guía completa
  - Arquitectura de integración
  - 3 opciones de integración explicadas
  - Implementación System Exec paso a paso

    - Ejemplos de código LabVIEW
    - Optimización y rendimiento
    - Troubleshooting completo

Tutorial

- examples/EJEMPLO_PASO_A_PASO.md - Tutorial práctico
    - 4 ejemplos incrementales
    - Código LabVIEW detallado
    - Aplicación de producción completa
    - Tips y debugging

🔌 Uso Básico desde LabVIEW

Detectar Puerto COM

```
[System Exec.vi]
  Command: python C:\MagicBridge\helpers\detect_port.py
  Output: "COM3"
```

Cargar Datos HMF

```
[System Exec.vi]
  Command: python C:\MagicBridge\bin\magicbridge-load.bat
           C:\Data\archivo.hmf COM3 5 D --quiet
  Return Code: 0 = éxito
```

Programar Chip

```
[System Exec.vi]
  Command: python C:\MagicBridge\bin\magicbridge-cmd.bat
           COM3 W5d --quiet
  Return Code: 0 = éxito
```

📊 Arquitectura

```
LabVIEW Application
        │
        ├─> System Exec.vi
        │   └─> Python Scripts (.pyc)
        │       └─> Serial Port (COM)
        │           └─> MagicBridge Hardware
        │
        └─> Parse Results (JSON/Text)
            └─> Update UI
```

💡 Ejemplo Completo (Pseudocódigo LabVIEW)

```
// Inicialización
puerto = DetectarPuerto()
CargarDatos("C:\Data\prod.hmf", puerto, 5, "D")

// Loop de producción
WHILE operando DO
    // Esperar chip
    MostrarMensaje("Coloque chip nuevo")

    // Programar
    resultado = EjecutarComando(puerto, "W5d")

    // Actualizar estadísticas
    IF resultado == OK THEN
        contador_ok++
    ELSE
        contador_error++
    END IF

    // Mostrar en pantalla
    ActualizarUI(contador_ok, contador_error)
END WHILE
```

🔧 Scripts Helper Incluidos

detect_port.py

Detecta automáticamente el puerto COM del MagicBridge.

Uso:

```
python detect_port.py
```

Salida:

```
COM3
```

magicbridge_api.py

API wrapper simplificada con interfaz CLI.

Uso:

```
REM Detectar puerto
python magicbridge_api.py detect

REM Cargar datos
python magicbridge_api.py load archivo.hmf COM3 5 D

REM Ejecutar comando
python magicbridge_api.py exec COM3 W5d
```

Salida: JSON estructurado

📋 Comandos Disponibles

Comando	Descripción	Ejemplo
W3d	Write PCB 3 variante D	`W3d`
W5d	Write PCB 5 variante D	`W5d`
W6d	Write PCB 6 variante D	`W6d`
R3d	Read PCB 3 variante D	`R3d`
R5d	Read PCB 5 variante D	`R5d`
P3d	Program PCB 3 variante D	`P3d`
P5d	Program PCB 5 variante D	`P5d`

Reemplazar 'd' con 'e' para variante E.

⚠️ Consideraciones Importantes

Timeouts

- Cargar datos: 300 segundos
- Programar chip: 60 segundos
- Leer chip: 30 segundos

Códigos de Salida

- `0`: Éxito
- `1`: Error en argumentos
- `2`: Error en archivo HMF
- `3`: Error de conexión
- `4`: Error en operación

Rutas de Archivos

Usar rutas absolutas en Windows:

```
C:\MagicBridge\...  ✅
.\MagicBridge\...   ❌
```

Modo Silencioso

Usar `--quiet` en producción para evitar output innecesario.

🐛 Troubleshooting

Error: "Python no encontrado"

Solución: Usar ruta completa

```
C:\Users\...\AppData\Local\Programs\Python\Python39\python.exe
```

Error: "Timeout"

Solución: Aumentar timeout en System Exec.vi a 300000 ms

Error: "Puerto COM no disponible"

Solución:

1. Verificar que MagicBridge esté conectado
2. Esperar 10 segundos después de conectar
3. Ejecutar detect_port.py para confirmar

VIs se cuelgan

Solución: Asegurar que:

1. `wait until completion` = TRUE en System Exec.vi
2. Timeout configurado correctamente
3. Scripts usan `--quiet` para evitar prompts

📞 Soporte

Ver documentación completa:

- INTEGRACION_LABVIEW.md - Guía completa
- EJEMPLO_PASO_A_PASO.md - Tutorial

✅ Checklist de Implementación

- [ ] Python instalado y en PATH

- [ ] pyserial instalado
- [ ] Scripts copiados a C:\MagicBridge\
- [ ] Helper scripts probados manualmente
- [ ] VI de detección de puerto creado y probado
- [ ] VI de carga de datos creado y probado
- [ ] VI de programación creado y probado
- [ ] Aplicación principal integrada
- [ ] Probado con chips reales
- [ ] Documentación de operador creada

MagicBridge + LabVIEW
Paquete de Integración v1.0
© 2026 LiCore